BOOTCAMP II ? ENTREGA INICIAL ? FASE 1

# Planejamento L?gico TaskTracker

Documento de arquitetura e receita l?gica para orientar a implementa??o em Python.

**Identifica??o acad?mica ? preencher antes da entrega**
Nome completo: _____
RA / matr?cula: _____ Curso/turma: _____
Polo: _____ E-mail institucional: _____

**Sobre este documento:** este planejamento foi elaborado agora para apoiar a realiza??o das Fases 1 e 2. A Fase 1 n?o havia sido realizada anteriormente; este arquivo n?o afirma entrega, aprova??o ou avalia??o acad?mica pr?via.

## 1. Identifica??o e entendimento do problema

### Contexto e problema

Na rotina de estudos, trabalho e atividades pessoais, ? comum acumular tarefas sem um lugar simples para registrar o que precisa ser feito. Anota??es espalhadas podem dificultar a consulta, a defini??o de prioridades e o acompanhamento de prazos. Uma tarefa esquecida ou sem informa??o clara pode atrasar outras atividades.

### Objetivo e solu??o proposta

O TaskTracker ser? um programa de linha de comando para cadastrar tarefas e consultar a lista durante sua execu??o. Para cada tarefa, o usu?rio informar? t?tulo, descri??o, prioridade e data limite. O sistema verificar? se o t?tulo foi preenchido e se a prioridade pertence ?s op??es permitidas. Cada registro come?ar? automaticamente com o status ?Pendente?.

### P?blico e limites

O prot?tipo poder? ser usado por estudantes e por qualquer pessoa que queira organizar tarefas de forma direta no terminal. Na Fase 2, os dados ficar?o em mem?ria enquanto o programa estiver aberto. N?o haver? banco de dados, persist?ncia, edi??o ou exclus?o. O objetivo ? praticar decomposi??o do problema, l?gica, Python e versionamento.

## 2. Decomposi??o do problema, dados e regras

### Entradas

| Dado | Uso |
|---|---|
| T?tulo | Identifica a tarefa. ? obrigat?rio; ser? armazenado sem espa?os extras nas extremidades. |
| Descri??o | Texto livre para detalhar a tarefa; poder? ficar vazio. |
| Prioridade | Alta, M?dia/Media ou Baixa; ser? padronizada ao armazenar. |
| Data limite | Texto informado para registrar a data planejada; sem valida??o avan?ada. |

### Sa?das

- Confirma??o quando uma tarefa for cadastrada.
- Mensagens de erro e novo pedido de entrada se t?tulo ou prioridade forem inv?lidos.
- Lista organizada com todas as propriedades das tarefas.
- Mensagem ?Nenhuma tarefa cadastrada no momento.? quando a lista estiver vazia.

### Regras

| Regra | Comportamento |
|---|---|
| T?tulo obrigat?rio | Rejeitar texto vazio ou composto somente de espa?os; repetir a solicita??o. |
| Prioridade restrita | Aceitar Alta, M?dia, Media ou Baixa sem diferenciar mai?sculas e min?sculas; repetir em caso de erro. |
| Descri??o e data | Descri??o livre e data limite em texto, sem regra de formato adicional. |
| Status inicial | Definir automaticamente ?Pendente? no cadastro. |
| Estrutura | Guardar cada tarefa em um dicion?rio e os dicion?rios em uma lista. |

O dicion?rio ter? as chaves `titulo`, `descricao`, `prioridade`, `data_limite` e `status`. A lista ser? criada ao iniciar; os dados n?o ser?o preservados depois do encerramento.

## 3. Receita l?gica ? algoritmo em portugu?s

1. Iniciar o programa.
2. Criar uma lista vazia chamada tarefas.
3. Exibir o menu: cadastrar tarefa, visualizar tarefas ou sair.
4. Ler a op??o escolhida.
5. Se a op??o for cadastrar, solicitar o t?tulo e remover espa?os das extremidades.
6. Enquanto o t?tulo estiver vazio, exibir erro e pedir outro t?tulo.
7. Solicitar a descri??o.
8. Solicitar a prioridade e comparar sem diferenciar mai?sculas e min?sculas com Alta, M?dia, Media e Baixa.
9. Enquanto a prioridade n?o for aceita, exibir erro e perguntar novamente. Armazenar a prioridade padronizada.
10. Solicitar a data limite e armazen?-la como texto.
11. Criar um dicion?rio com t?tulo, descri??o, prioridade, data limite e status ?Pendente?.
12. Adicionar o dicion?rio ? lista e confirmar o cadastro.
13. Se a op??o for visualizar, verificar se a lista est? vazia. Se estiver, avisar; caso contr?rio, percorrer a lista e exibir todas as propriedades.
14. Se a op??o n?o for cadastrar, visualizar nem sair, informar que ela ? inv?lida.
15. Depois de cadastrar, listar ou mostrar erro, retornar ao menu. Encerrar apenas quando o usu?rio escolher sair.

**Estrutura prevista para cada registro**

```
tarefa = {
    "titulo": titulo,
    "descricao": descricao,
    "prioridade": prioridade,
    "data_limite": data_limite,
    "status": "Pendente"
}
```

# 4. Mapeamento arquitetural ? Git e GitHub

**Git** ? um sistema local de controle de vers?o que registra mudan?as do projeto. Cada commit guarda uma etapa coerente com uma mensagem que explica seu prop?sito, permitindo acompanhar a evolu??o do trabalho.

**GitHub** hospeda o reposit?rio Git remotamente, para publicar c?digo e documentos. A branch principal planejada ? `main`; o envio ao GitHub ser? realizado ap?s autoriza??o do estudante.

### Estrutura planejada para a Fase 2

```
tasktracker-projeto/
??? README.md
??? .gitignore
??? docs/
?   ??? planejamento_logico.pdf
??? src/
    ??? main.py
```

O README apresenta a finalidade e instru??es; o .gitignore ignora arquivos tempor?rios; o PDF guarda este planejamento; e `src/main.py` conter? a aplica??o CLI em Python.

### Etapas de versionamento previstas

1. Registrar estrutura, README inicial, .gitignore e planejamento.
2. Adicionar menu e estrutura inicial do programa.
3. Adicionar cadastro, valida??es e listagem.
4. Revisar implementa??o e documenta??o.

Ap?s os commits locais, ser?o conferidos branch, hist?rico e estado do reposit?rio. Com autoriza??o, ser? criado um reposit?rio p?blico e enviado o hist?rico da branch `main`. Os PDFs de refer?ncia e credenciais ficam fora do reposit?rio.

# 5. Link da apresenta??o em v?deo

**Link do v?deo: PENDENTE ? preencher ap?s a grava??o**

O enunciado prev? um v?deo de 5 minutos no YouTube, horizontal (16:9), com o estudante em c?mera e ? udio claro. A visibilidade deve ser P?blico ou N?o Listado. O formato Shorts/vertical, v?deo privado e grava??o apenas da tela sem o rosto n?o atendem ?s instru??es. A Fase 2 tamb?m requer voz pr?pria.

## Apresenta??o prevista

O estudante poder? apresentar sua identifica??o acad?mica, explicar o problema, mostrar este documento, percorrer o algoritmo e comentar a estrutura planejada para GitHub. O v?deo ainda n?o foi gravado nem publicado.

## Continuidade entre fases

| Fase | Continuidade |
|---|---|
| Fase 1 ? Planejamento l?gico (10 pontos) | Definir problema, dados, regras, algoritmo, estrutura Git/GitHub e preparar apresenta??o. |
| Fase 2 ? Entrega intermedi?ria (15 pontos) | Usar o planejamento, criar reposit?rio, implementar CLI em Python, validar regras, registrar commits e demonstrar o c?digo. |
| Fase 3 ? Entrega final (25 pontos) | Etapa futura, fora do escopo atual. Docker e deploy n?o ser?o implementados. |

Este documento orienta o desenvolvimento. Entrega e avalia??o dependem da submiss?o e dos crit?rios do curso.